

Code :

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

import joblib

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder

from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


print("="*60)

print("GENERATING DATASET...")

print("="*60)

np.random.seed(42)

n = 2000

education_levels = ["Bachelor's", "Master's", "PhD"]

job_roles = [

    "Developer",

    "Data Analyst",

    "Manager",

    "Designer",

    "Data Scientist"

]

company_types = ["Startup", "MNC", "Medium Scale"]

genders = ["Male", "Female"]

df = pd.DataFrame({

    "EmployeeID": range(1, n + 1),
```

```
"Age": np.random.randint(21, 60, n),
"Gender": np.random.choice(genders, n),
"EducationLevel": np.random.choice(education_levels, n),
"YearsOfExperience": np.random.randint(0, 25, n),
"JobRole": np.random.choice(job_roles, n),
"SkillsScore": np.random.randint(1, 11, n),
"Certifications": np.random.randint(0, 10, n),
"CompanyType": np.random.choice(company_types, n),
"WorkHoursPerWeek": np.random.randint(30, 60, n)
})
```

```
salary = (
    25000
    + df["YearsOfExperience"] * 5000
    + df["SkillsScore"] * 3000
    + df["Certifications"] * 1500
)
```

```
salary += df["EducationLevel"].map({
    "Bachelor's": 0,
    "Master's": 10000,
    "PhD": 25000
})
```

```
salary += df["JobRole"].map({
    "Developer": 10000,
    "Data Analyst": 8000,
    "Designer": 7000,
```

```

    "Manager": 20000,
    "Data Scientist": 18000
})

salary += np.random.normal(0, 5000, n)
df["Salary"] = salary.astype(int)

# Save dataset
df.to_csv("employee_salary_dataset.csv", index=False)
print("Dataset Shape:", df.shape)
print("Dataset Saved as employee_salary_dataset.csv")
print("\n" + "="*60)
print("DATA PREPROCESSING")
print("="*60)

for col in ["Gender", "EducationLevel"]:
    df.loc[df.sample(frac=0.01).index, col] = np.nan

print("\nMissing Values Before:")
print(df.isnull().sum())

# Handle missing values
df.fillna(df.mode().iloc[0], inplace=True)

# Remove duplicates
df.drop_duplicates(inplace=True)
print("\nMissing Values After:")
print(df.isnull().sum())

# Label Encoding
encoders = {}
categorical_columns = [
    "Gender",

```

```

    "EducationLevel",
    "JobRole",
    "CompanyType"
]

for col in categorical_columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    encoders[col] = le

print("\nPreprocessing Completed!")
print("\n" + "="*60)
print("EDA")
print("="*60)
print(df.describe())
plt.figure(figsize=(8,6))
sns.scatterplot(
    x=df["YearsOfExperience"],
    y=df["Salary"]
)
plt.title("Experience vs Salary")
plt.show()

# Average Salary by Job Role
temp = pd.read_csv("employee_salary_dataset.csv")
plt.figure(figsize=(8,6))
temp.groupby("JobRole")["Salary"].mean().sort_values().plot(
    kind="bar",
    color="skyblue"

```

```
)  
plt.title("Average Salary by Job Role")  
plt.ylabel("Average Salary")  
plt.show()
```

```
# Correlation Heatmap
```

```
plt.figure(figsize=(10,6))  
sns.heatmap(  
    df.corr(numeric_only=True),  
    annot=True,  
    cmap="coolwarm"  
)  
plt.title("Correlation Heatmap")  
plt.show()
```

```
# Salary Distribution by Education
```

```
plt.figure(figsize=(8,6))  
sns.boxplot(  
    x=temp["EducationLevel"],  
    y=temp["Salary"]  
)  
plt.title("Salary Distribution by Education Level")  
plt.show()  
print("\n" + "="*60)  
print("MODEL TRAINING")  
print("="*60)  
X = df.drop(["Salary", "EmployeeID"], axis=1)  
y = df["Salary"]
```

```

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

model = RandomForestRegressor(
    n_estimators=200,
    random_state=42
)

model.fit(X_train, y_train)

predictions = model.predict(X_test)
mae = mean_absolute_error(y_test, predictions)
rmse = np.sqrt(mean_squared_error(y_test, predictions))
r2 = r2_score(y_test, predictions)

print("\nMODEL PERFORMANCE")
print("-"*40)
print(f"MAE : {mae:.2f}")
print(f"RMSE : {rmse:.2f}")
print(f"R2 : {r2:.4f}")
importance = pd.Series(
    model.feature_importances_,
    index=X.columns
)

```

```
).sort_values()
```

```
plt.figure(figsize=(8,6))
```

```
importance.plot(kind="barh", color="green")
```

```
plt.title("Feature Importance")
```

```
plt.show()
```

```
joblib.dump(model, "salary_prediction_model.pkl")
```

```
print("\nModel Saved as salary_prediction_model.pkl")
```

```
print("\n" + "="*60)
```

```
print("EMPLOYEE SALARY PREDICTION")
```

```
print("="*60)
```

```
try:
```

```
    age = int(input("Enter Age: "))
```

```
    education = input(
```

```
        "Education (Bachelor's/Master's/PhD): "
```

```
    )
```

```
    experience = int(
```

```
        input("Years of Experience: ")
```

```
    )
```

```
    skills = int(
```

```
        input("Skills Score (1-10): ")
```

```
    )
```

```
certifications = int(
    input("Number of Certifications: ")
)

role = input(
    "Job Role (Developer/Data Analyst/Manager/Designer/Data Scientist): "
)

company = input(
    "Company Type (Startup/MNC/Medium Scale): "
)

gender = input(
    "Gender (Male/Female): "
)

work_hours = int(
    input("Work Hours Per Week: ")
)

input_df = pd.DataFrame([
    "Age": age,
    "Gender": encoders["Gender"].transform([gender])[0],
    "EducationLevel": encoders["EducationLevel"].transform([education])[0],
    "YearsOfExperience": experience,
    "JobRole": encoders["JobRole"].transform([role])[0],
    "SkillsScore": skills,
    "Certifications": certifications,
```

```
"CompanyType": encoders["CompanyType"].transform([company])[0],  
"WorkHoursPerWeek": work_hours  
}})
```

```
predicted_salary = model.predict(input_df)[0]
```

```
print("\n" + "="*60)  
print(f"Predicted Employee Salary: ₹{predicted_salary:,.2f}")  
print("="*60)
```

```
except Exception as e:
```

```
    print("\nInvalid Input!")  
    print("Error:", e)
```

```
print("\nPROJECT COMPLETED SUCCESSFULLY!")
```

output:

```
=====
```

```
GENERATING DATASET...
```

```
=====
```

```
Dataset Shape: (2000, 11)
```

```
Dataset Saved as employee_salary_dataset.csv
```

```
=====
```

```
DATA PREPROCESSING
```

```
=====
```

Missing Values Before:

EmployeeID	0
Age	0
Gender	20
EducationLevel	20
YearsOfExperience	0
JobRole	0
SkillsScore	0
Certifications	0
CompanyType	0
WorkHoursPerWeek	0
Salary	0

dtype: int64

Missing Values After:

EmployeeID	0
Age	0
Gender	0
EducationLevel	0
YearsOfExperience	0
JobRole	0
SkillsScore	0
Certifications	0
CompanyType	0
WorkHoursPerWeek	0
Salary	0

dtype: int64

Preprocessing Completed!

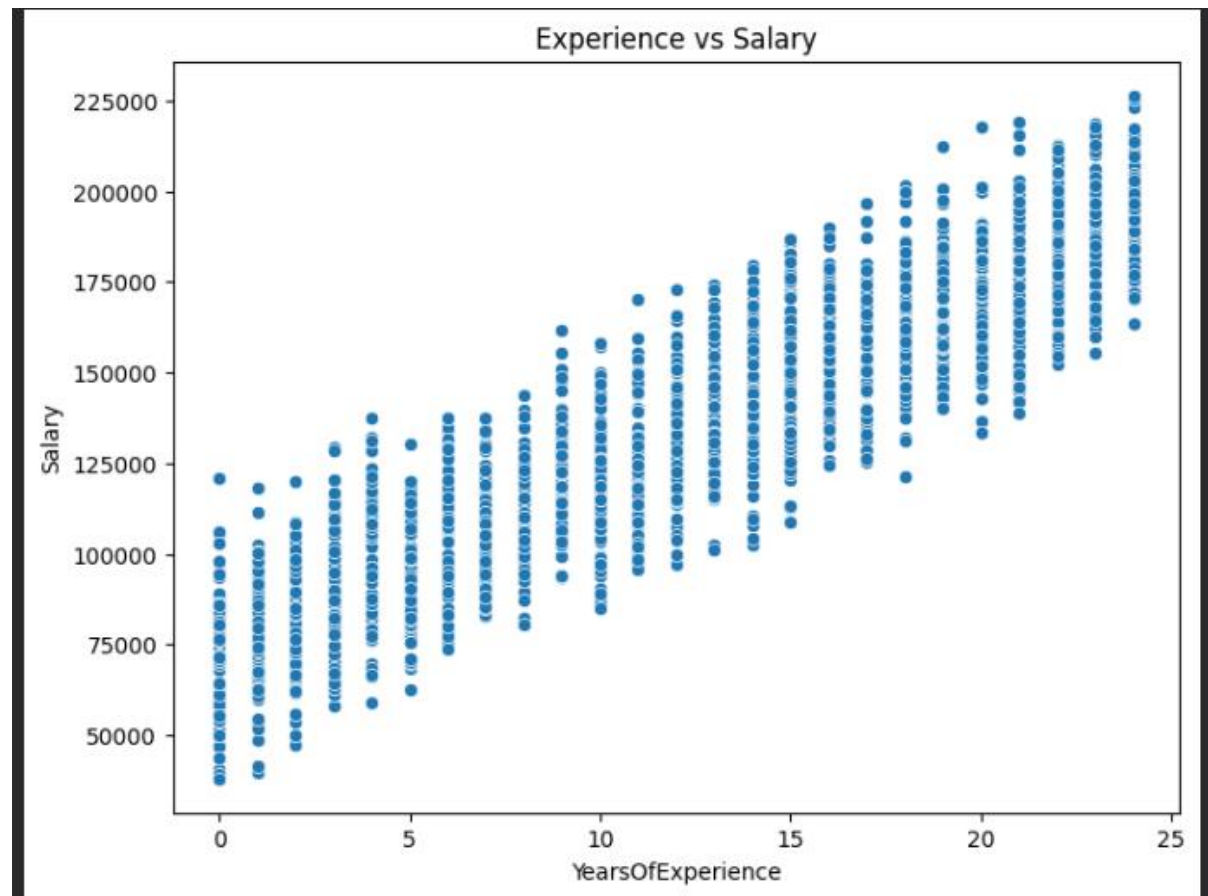
EDA

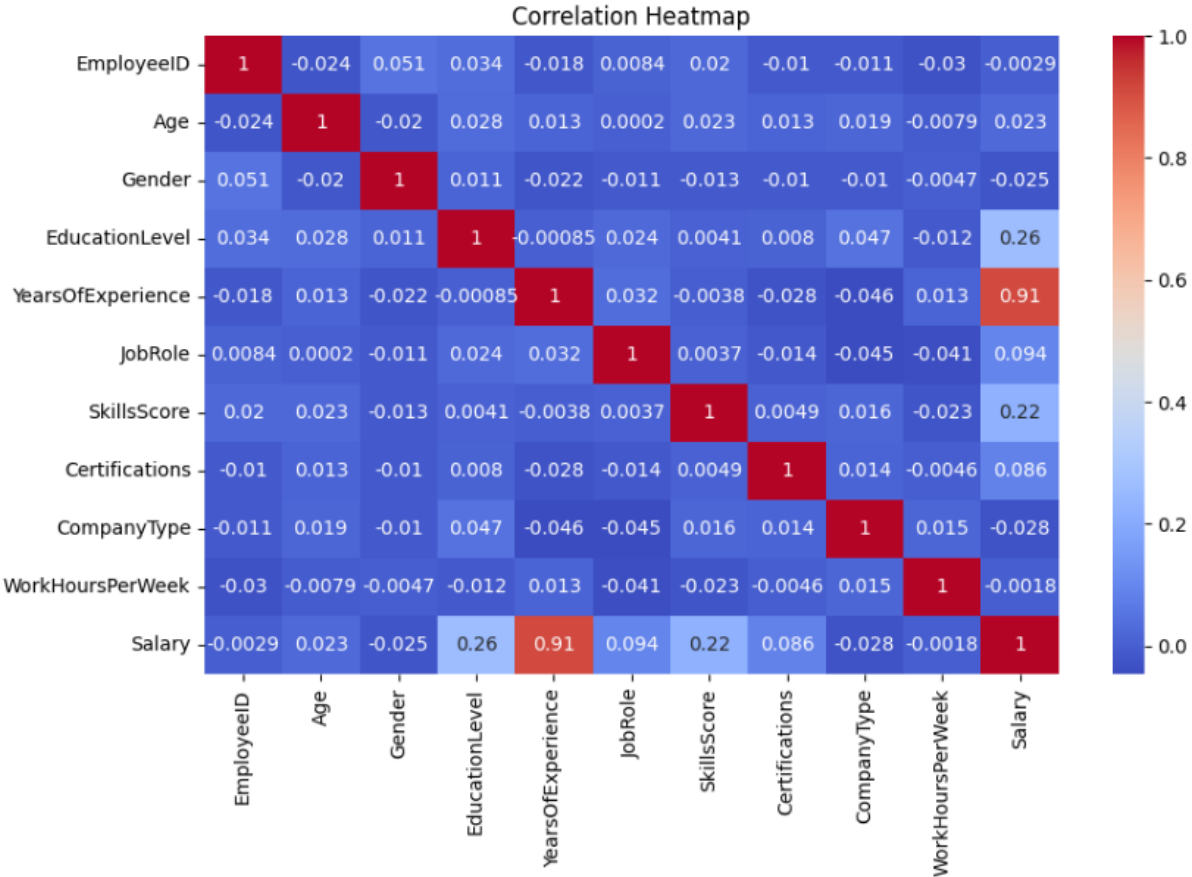
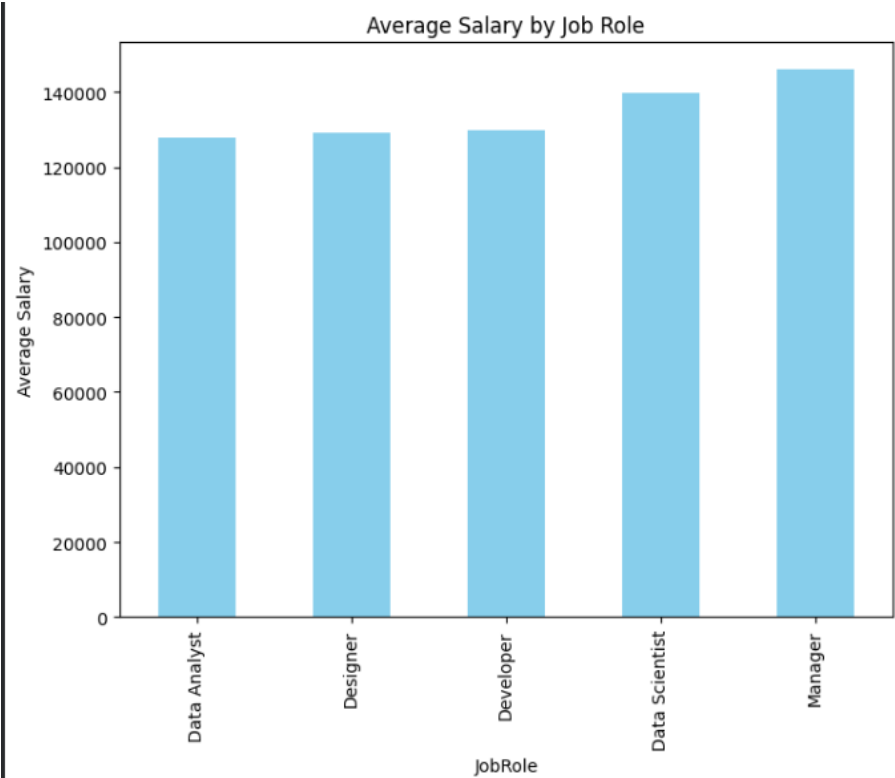
	EmployeeID	Age	Gender	EducationLevel \
count	2000.000000	2000.000000	2000.000000	2000.000000
mean	1000.500000	40.358500	0.505500	1.010500
std	577.494589	11.359379	0.500095	0.822635
min	1.000000	21.000000	0.000000	0.000000
25%	500.750000	30.000000	0.000000	0.000000
50%	1000.500000	41.000000	1.000000	1.000000
75%	1500.250000	50.000000	1.000000	2.000000
max	2000.000000	59.000000	1.000000	2.000000

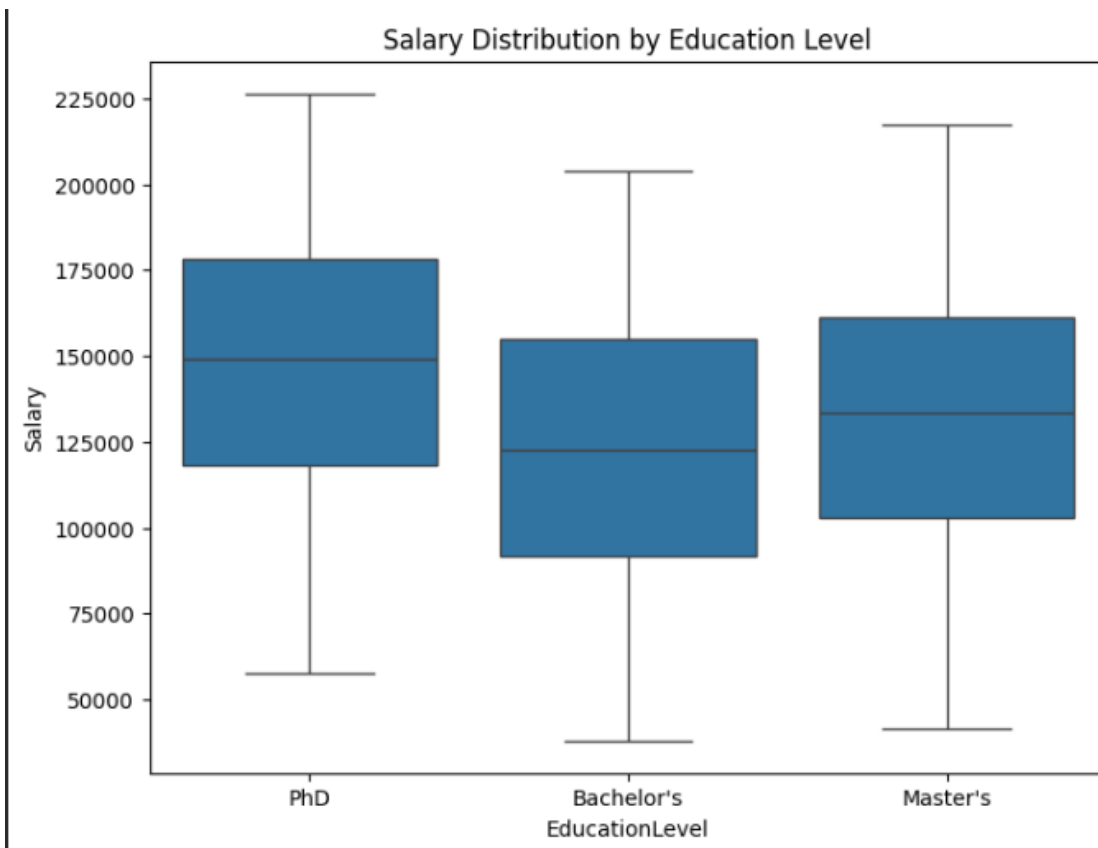
	YearsOfExperience	JobRole	SkillsScore	Certifications \
count	2000.000000	2000.000000	2000.000000	2000.000000
mean	12.338000	1.946500	5.537500	4.554500
std	7.190244	1.422199	2.940224	2.854138
min	0.000000	0.000000	1.000000	0.000000
25%	6.000000	1.000000	3.000000	2.000000
50%	12.000000	2.000000	6.000000	5.000000
75%	19.000000	3.000000	8.000000	7.000000
max	24.000000	4.000000	10.000000	9.000000

	CompanyType	WorkHoursPerWeek	Salary
count	2000.000000	2000.000000	2000.000000
mean	0.967500	44.488000	134219.815500

std	0.813495	8.774586	39351.652742
min	0.000000	30.000000	38003.000000
25%	0.000000	37.000000	103231.000000
50%	1.000000	45.000000	134944.500000
75%	2.000000	52.000000	165272.500000
max	2.000000	59.000000	226426.000000







=====

MODEL TRAINING

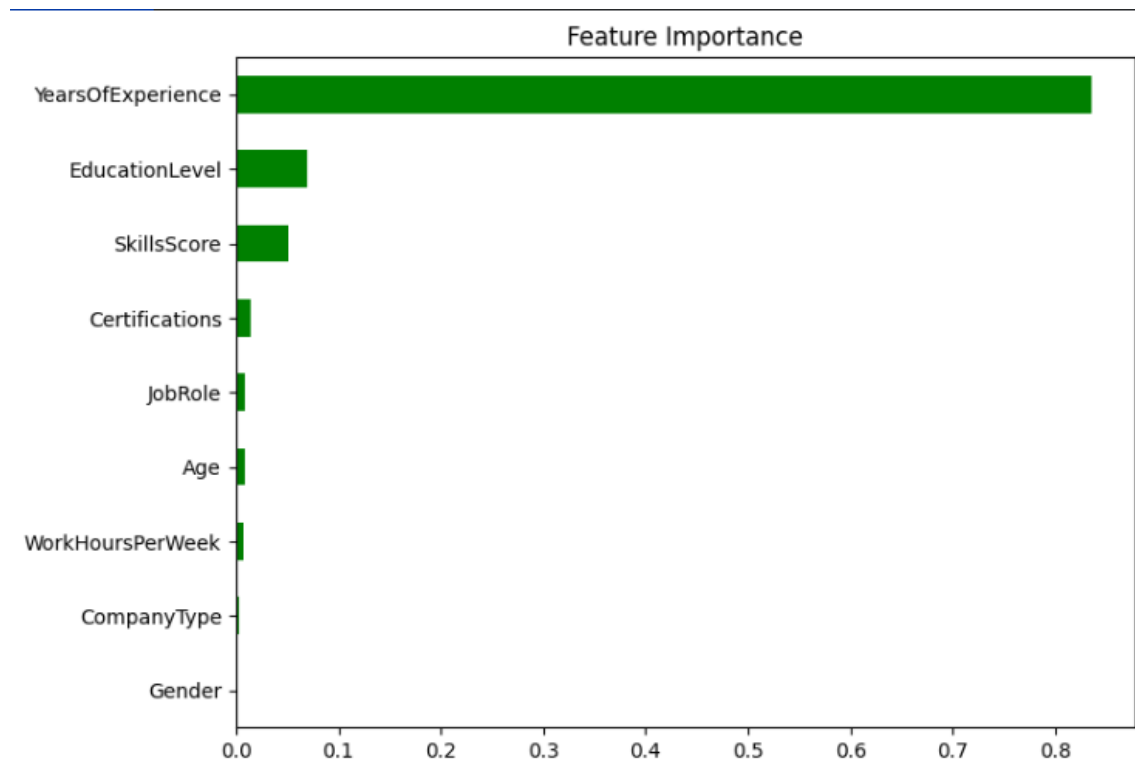
=====

MODEL PERFORMANCE

MAE : 6826.55

RMSE : 8624.59

R² : 0.9531



Model Saved as salary_prediction_model.pkl

```
=====
```

EMPLOYEE SALARY PREDICTION

```
=====
```

Enter Age: 19

Education (Bachelor's/Master's/PhD): Bachelor's

Years of Experience: 2 years

Invalid Input!

Error: invalid literal for int() with base 10: '2 years'

PROJECT COMPLETED SUCCESSFULLY!